

ECS 160 – Fuzzing, Design Patterns

Instructor: Tapti Palit

Teaching Assistant: **Xingming Xu**



Agenda

- **Fuzzing**
- Sanitizing
- Structural design patterns
 - Composite
 - Decorator
 - Bridge



Recall: Unit Tests and Integration Tests

- Unit tests:
 - We're testing individual functions here
 - Manually add assertions, identify corner cases, etc.
- Integration tests:
 - Manually test end-to-end functionality



Fuzzing

- Automated testing with **unexpected, weird, malformed inputs**
- Look for crashing/weird behavior
 - So we can consider this to be for the *entire application*
- Generate stuff until we screw up
- This works well with interfaces accepting complex input
- Fuzzing can be done in any language

High-level Idea

- Where do we fuzz?
 - Input boundaries
 - Serialization
 - Utility libraries
- Approach: Mutation fuzzing
- Approach: Coverage-guided fuzzing



Mutation Fuzzing: Overall Approach

- Start with valid input
- Use seeds to transform inputs
- Run
- Watch for crashes



Mutation Fuzzing: Seed Corpus

- Transformation is only as effective as seed corpus
- If we have purely random examples, they're not necessarily good inputs!
 - A random input may not even allow relevant code to be tested



Mutation Fuzzing: Strategies

- Byte-transformations
 - Shuffle bytes
 - Flip byte
 - Erase/zero out entire regions
 - Insert bytes
- Structural mutations
 - “Insert garbage” into string
 - “Random replacements/removals”



Mutation Fuzzing: Strategies

- Dictionaries
 - Messing around with keyword dictionary
 - SQL:
 - SELECT, FROM, WHERE, GROUP BY
- Edge cases (pretty effective!)
 - Empty string, long strings, etc.
 - Deeply recursive serialization



Coverage

- Mutators produce many invalid inputs
- We pick “good seeds” that will maximize the metric: **coverage**
 - Amount of code or execution paths explored during mutation-fuzz testing
 - Finding reward signals: think of a simple RL agent...
- State of the art approach

Coverage

- High coverage indicates high potential
 - Thus, we want to reward those reward seeds
- Record the coverage
- Prioritize those seeds that give higher coverage
- Store corpus of seeds that observed high coverage



Reinforcement Learning and Coverage

- Even if we're not learning a neural network, we're doing something quite similar
- Reward: code with new coverage
- Policy: mutate inputs that earned reward
- State: program state reached -> crash is a **terminal state**
- Action: mutations to input
- Policy update: keep mutating rewarding inputs, or mutate them more

Agenda

- Fuzzing
- **Sanitizing**
- Structural design patterns
 - Composite
 - Decorator
 - Bridge



Sanitizing

- Provides an additional layer of protection against undefined behavior
- Memory safety bugs may or may not actually crash the program, which is what fuzzing is meant to detect
- Introduce sanitizers – additional compiler instrumentation

Sanitizers

- Crash on otherwise silent undefined behavior
- Popular sanitizers
 - Address sanitizer (ASAN) – can detect buffer overflow
 - Leak sanitizer (LSAN) – can detect memory leak
 - Thread sanitizer (TSAN) – can detect data races
- These are additional instructions to the compiler, so they slow down execution
- NOTE: Java does not have undefined behavior in the C++ sense, C++ needs sanitizers with fuzzing to help make memory issues visible

Agenda

- Fuzzing
- Sanitizing
- **Structural design patterns**
 - Composite
 - Decorator
 - Bridge



Structural Design Patterns

- Proposed solution patterns to “how we compose classes and objects”
 - Concerned with object relationships



Composite

- Important characteristics:
 - Tree structure
 - Treat individual objects, groups uniformly
- Good for hierarchy, reducing code duplication



Composite

```
public abstract class SocialMediaComponent {
    public abstract void show();
    public abstract void hide();
    public abstract void report();
}

public class Post extends SocialMediaComponent {
}

public class Thread extends SocialMediaComponent
{}

public class Space extends SocialMediaComponent
{
    public void show() { // show livestream }
    public void hide() { // hide livestream }
    public void report() { // report livestream}
}

class ShowButton {
    SocialMediaComponent content;
    void onClick () { content.show(); }
}
```



Composite

- In the previous example, `show()`, `hide()`, `report()` all exist in the abstract `SocialMediaComponent`
- Our `Posts`, `Threads`, `Space` extending that can all call the same methods without duplicating them in each class

Decorator

- Adding behaviors to objects dynamically
- Simple example:

```
InputStream in = new BufferedInputStream(  
    new FileInputStream("data.txt"));
```

- Combine behaviors by wrapping
 - Both BufferedInputStream and FileInputStream inherit from InputStream



Decorator

- Consider using when you need features “as you go”
 - More flexible than subclassing, and easier to add functionality
- Difficult to debug



Decorators vs Proxies

- Proxies and decorators have different responsibilities, despite being implementation-wise similar
- Decorators add responsibilities; proxies control access



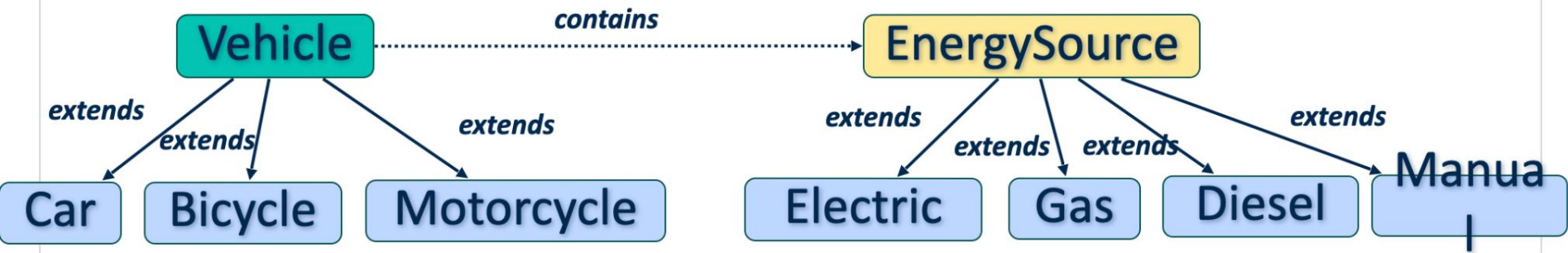
Bridge

- Decouple abstraction from implementation
 - Separating an abstraction from its implementation
 - Both can vary independently
- Two dimensions, abstraction + independence, that vary independently



Bridge

- This gets rid of deep, complex, intertwining inheritance trees



Bridge

- Since they're decoupled, we don't have to make a cross product of all possible classes to exist
- Easier to accommodate our 2 hierarchies

